LAPORAN UTS FRAMEWORK BASED PROGRAMMING

Pengembangan Web Aplikasi Pengelolaan dan Pembagian Anggaran secara Dinamis (Smart Budget Allocation Planner)

Dimas Sahputra 092023090193

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK & ILMU KOMPUTER

JAKARTA GLOBAL UNIVERSITY

2026

1. Pendahuluan

1.1 Latar Belakang

Pengelolaan anggaran membutuhkan perencanaan yang baik agar pembagian dana sesuai dengan kebutuhan. Proses pengelolaan secara manual dapat menyebabkan kesalahan perhitungan dan kesulitan dalam memantau alokasi pengeluaran.

Oleh karena itu, dikembangkan aplikasi **Smart Budget Allocation Planner**, yaitu aplikasi berbasis web yang membantu pengguna mengatur anggaran, membuat kategori pengeluaran, mengelola rincian sub-item, serta melakukan perhitungan dan validasi anggaran secara otomatis menggunakan framework Angular.

1.2 Rumusan Masalah

1. Bagaimana merancang aplikasi pengelolaan anggaran menggunakan Angular?
2. Bagaimana mengelola kategori dan sub-item secara dinamis?
3. Bagaimana menerapkan validasi dan perhitungan anggaran secara otomatis?
4. Bagaimana menampilkan ringkasan anggaran secara dinamis?

1.3 Tujuan

1. Membangun aplikasi Smart Budget Allocation Planner berbasis Angular.
2. Mengimplementasikan Reactive Forms, FormArray, dan Nested FormArray.
3. Menerapkan validasi dan perhitungan alokasi anggaran secara otomatis.
4. Mengimplementasikan komunikasi antar-komponen menggunakan Angular.

1.4 Manfaat

Aplikasi ini diharapkan dapat membantu pengguna merencanakan dan memantau alokasi anggaran dengan lebih mudah, serta menjadi sarana penerapan konsep Framework Based Programming menggunakan Angular.

1.5 Batasan Masalah

1. Aplikasi dikembangkan menggunakan Angular, TypeScript, HTML, dan CSS.
2. Pengembangan difokuskan pada View dan Controller tanpa backend maupun database.
3. Pengelolaan data menggunakan Angular Reactive Forms.
4. Aplikasi memiliki fitur pengelolaan anggaran, kategori, dan sub-item.
5. Data hanya disimpan sementara selama aplikasi berjalan.

2. ANALISIS DAN PERANCANGAN SISTEM

2.1 Gambaran Umum Sistem

Smart Budget Allocation Planner merupakan aplikasi berbasis web yang dirancang untuk membantu pengguna dalam mengelola dan merencanakan pembagian anggaran. Pengguna dapat memasukkan total anggaran, membuat kategori pengeluaran, menentukan persentase alokasi, serta menambahkan rincian sub-item pada setiap kategori.

Sistem melakukan perhitungan nominal alokasi secara otomatis berdasarkan persentase kategori dan menyediakan validasi untuk memastikan pembagian anggaran sesuai dengan ketentuan. Hasil pengelolaan ditampilkan dalam bentuk ringkasan anggaran yang diperbarui secara dinamis.

2.2 Identifikasi Aktor

Aplikasi Smart Budget Allocation Planner memiliki satu aktor utama, yaitu:

Aktor	**Deskripsi**
Pengelola Anggaran	Pengguna yang bertanggung jawab menginput total anggaran, mengelola kategori, mengatur rincian sub-item, dan melihat ringkasan anggaran.

2.3 Identifikasi Use Case

Berdasarkan kebutuhan sistem, terdapat delapan use case yang menggambarkan fungsi utama aplikasi.

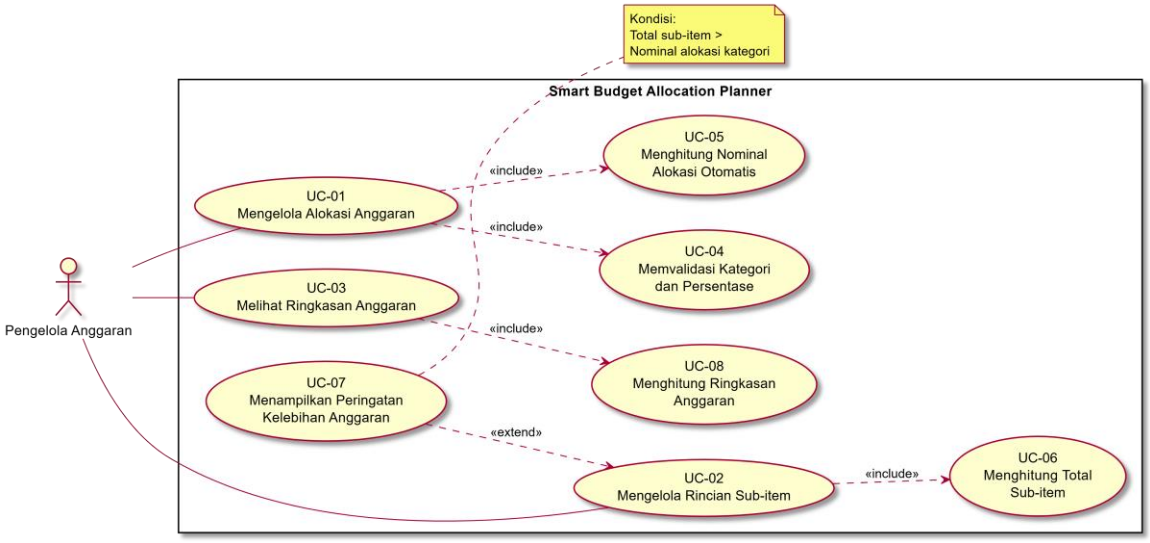
ID	**Nama Use Case**	**Deskripsi**
UC-01	Mengelola Alokasi Anggaran	Mengatur total anggaran dan persentase alokasi setiap kategori.
UC-02	Mengelola Rincian Sub-item	Menambahkan dan menghapus rincian pengeluaran pada setiap kategori.

UC-03	Melihat Ringkasan Anggaran	Menampilkan hasil perhitungan dan kondisi keseluruhan anggaran.
UC-04	Memvalidasi Kategori dan Persentase	Memeriksa nama kategori dan kesesuaian total persentase alokasi.
UC-05	Menghitung Nominal Alokasi Otomatis	Menghitung nominal anggaran berdasarkan persentase kategori.
UC-06	Menghitung Total Sub-item	Menghitung keseluruhan nominal rincian pengeluaran pada kategori.
UC-07	Menampilkan Peringatan Kelebihan Anggaran	Menampilkan peringatan ketika total sub-item melebihi nominal alokasi kategori.
UC-08	Menghitung Ringkasan Anggaran	Menghitung total alokasi, total sub-item, dan sisa anggaran.

2.4 Use Case Diagram

Use Case Diagram digunakan untuk menggambarkan hubungan antara aktor dengan fungsi-fungsi yang tersedia pada aplikasi Smart Budget Allocation Planner.

Diagram ini menunjukkan bahwa Pengelola Anggaran dapat mengelola alokasi anggaran, mengelola rincian sub-item, dan melihat ringkasan anggaran. Beberapa proses pendukung menggunakan relasi `<<include>>`, sedangkan peringatan kelebihan anggaran menggunakan relasi `<<extend>>` karena hanya ditampilkan ketika kondisi tertentu terpenuhi.



Gambar 2.1 Use Case Diagram Smart Budget Allocation Planner

2.5 Deskripsi Singkat Alur Sistem

Secara umum, proses penggunaan aplikasi dimulai ketika Pengelola Anggaran memasukkan total anggaran dan menentukan kategori pengeluaran beserta persentase alokasinya.

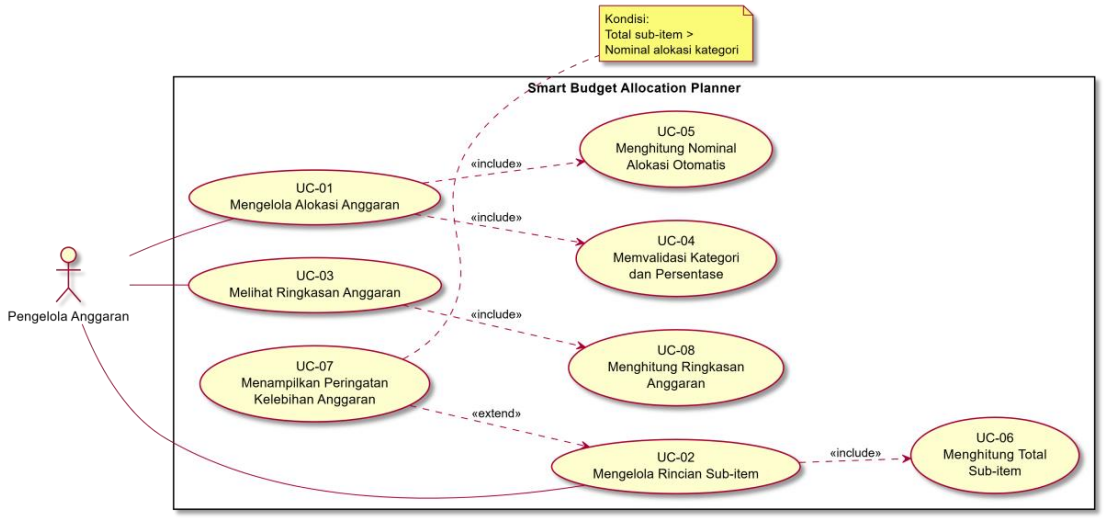
Selanjutnya, sistem melakukan validasi kategori dan menghitung nominal alokasi secara otomatis. Pengguna dapat menambahkan rincian sub-item pada setiap kategori. Sistem kemudian menghitung total sub-item dan memberikan peringatan apabila jumlahnya melebihi nominal alokasi kategori.

Setelah data dimasukkan, sistem menampilkan ringkasan anggaran yang berisi total anggaran, total alokasi, total sub-item, serta sisa anggaran.

3. USE CASE DESCRIPTION & SEQUENCE DIAGRAM

3.1 Use Case Description

Use Case Description merupakan penjelasan rinci mengenai interaksi antara aktor dan sistem dalam menjalankan suatu fungsi. Pada aplikasi Smart Budget Allocation Planner, terdapat tiga use case utama yang dijelaskan secara rinci, yaitu mengelola alokasi anggaran, mengelola rincian sub-item, dan melihat ringkasan anggaran.



Gambar 3.1 Use Case Diagram Smart Budget Allocation Planner

3.1.1 UC-01 — Mengelola Alokasi Anggaran

Use case ini menjelaskan proses pengguna dalam mengatur total anggaran serta menentukan kategori dan persentase alokasi pengeluaran.

- **Aktor:** Pengelola Anggaran.
- **Tujuan:** Mengatur pembagian total anggaran ke dalam beberapa kategori pengeluaran.
- **Precondition:** Pengguna telah membuka aplikasi Smart Budget Allocation Planner.

Main Flow:

1. Pengguna memasukkan total anggaran.

2. Sistem menampilkan form pengelolaan anggaran.
3. Pengguna menambahkan kategori anggaran.
4. Pengguna mengisi nama dan persentase kategori.
5. Sistem memvalidasi kategori dan persentase.
6. Sistem menghitung nominal alokasi secara otomatis.
7. Pengguna dapat menambah atau menghapus kategori.
8. Sistem memperbarui hasil perhitungan anggaran.

Alternative Flow:

- Nama kategori kosong atau duplikat.
- Total persentase tidak sesuai 100%.
- Jumlah kategori telah mencapai batas maksimal.
- Data input tidak memenuhi ketentuan validasi.

Postcondition: Data kategori dan nominal alokasi anggaran diperbarui sesuai input pengguna.

3.1.2 UC-02 — Mengelola Rincian Sub-item

Use case ini menjelaskan proses pengguna dalam menambahkan dan menghapus rincian pengeluaran pada setiap kategori anggaran.

- **Aktor:** Pengelola Anggaran.
- **Tujuan:** Mengelola rincian penggunaan dana pada setiap kategori pengeluaran.
- **Precondition:** Pengguna telah membuat kategori anggaran.

Main Flow:

1. Pengguna memilih kategori anggaran.
2. Sistem menampilkan rincian sub-item.
3. Pengguna menambahkan sub-item.
4. Pengguna mengisi nama dan nominal sub-item.
5. Sistem menghitung total sub-item.
6. Sistem memvalidasi total sub-item terhadap nominal alokasi kategori.
7. Pengguna dapat menambah atau menghapus sub-item.
8. Sistem memperbarui subtotal dan status validasi.

Alternative Flow:

- Nama sub-item kosong atau duplikat dalam kategori yang sama.
- Nominal sub-item tidak valid.
- Total sub-item melebihi nominal alokasi kategori.
- Pengguna menghapus sub-item yang telah dimasukkan.

Postcondition: Data rincian pengeluaran dan total sub-item diperbarui sesuai perubahan pengguna.

3.1.3 UC-03 — Melihat Ringkasan Anggaran

Use case ini menjelaskan proses sistem dalam menampilkan hasil perhitungan keseluruhan anggaran berdasarkan data yang telah dimasukkan pengguna.

- **Aktor:** Pengelola Anggaran.
- **Tujuan:** Mengetahui kondisi keseluruhan pembagian dan penggunaan anggaran.
- **Precondition:** Pengguna telah membuka aplikasi.

Main Flow:

1. Sistem mengambil data anggaran dan kategori.
2. Sistem menghitung total persentase alokasi.
3. Sistem menghitung total nominal alokasi.
4. Sistem menghitung total seluruh sub-item.
5. Sistem menghitung sisa anggaran dan sisa alokasi detail.
6. Sistem memeriksa status validasi anggaran.
7. Sistem menampilkan ringkasan anggaran kepada pengguna.

Alternative Flow:

- Belum terdapat kategori anggaran.
- Data anggaran belum lengkap.
- Total persentase belum mencapai 100%.
- Terdapat kategori dengan total sub-item melebihi nominal alokasinya.

Postcondition: Ringkasan dan status anggaran ditampilkan berdasarkan kondisi data terbaru.

3.2 Sequence Diagram

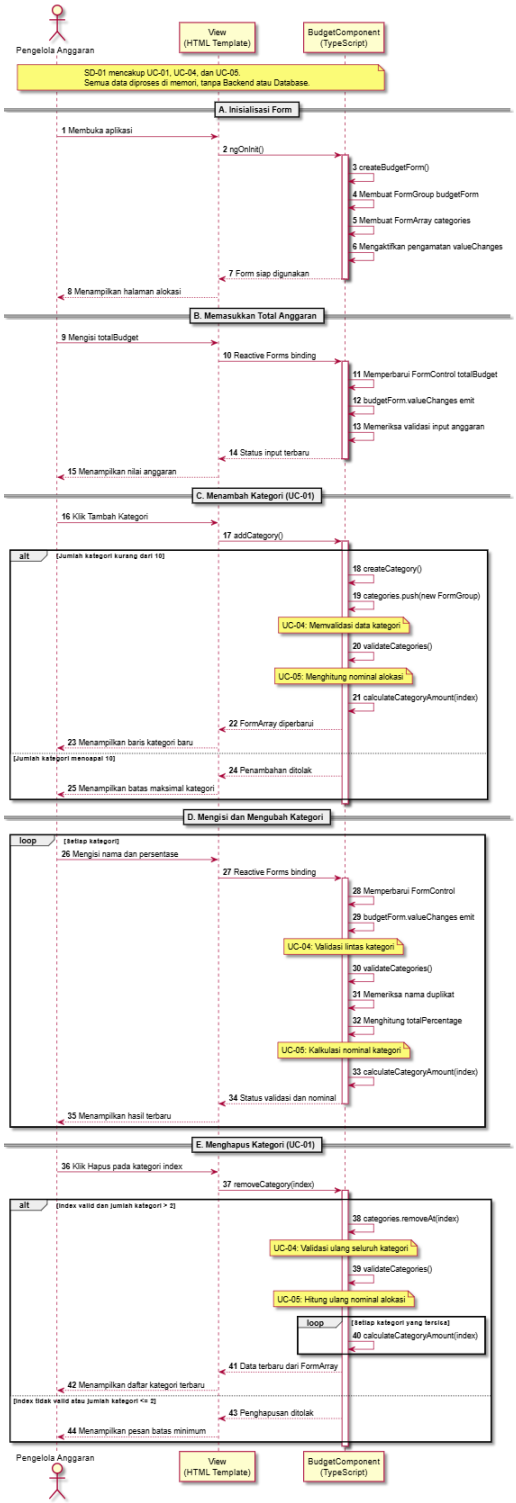
Sequence Diagram digunakan untuk menggambarkan urutan interaksi antara aktor, komponen, dan proses sistem berdasarkan waktu. Diagram ini memperlihatkan bagaimana suatu permintaan pengguna diterima, diproses, divalidasi, dan menghasilkan respons pada aplikasi.

Pada aplikasi Smart Budget Allocation Planner, terdapat tiga Sequence Diagram yang menggambarkan proses utama sistem.

3.2.1 SD-01 — Mengelola Alokasi Anggaran

Sequence Diagram ini menggambarkan interaksi pengguna dengan sistem ketika memasukkan total anggaran, menambahkan atau menghapus kategori, serta menentukan persentase alokasi.

Sistem kemudian menjalankan validasi kategori dan menghitung nominal alokasi secara otomatis berdasarkan total anggaran dan persentase yang dimasukkan.

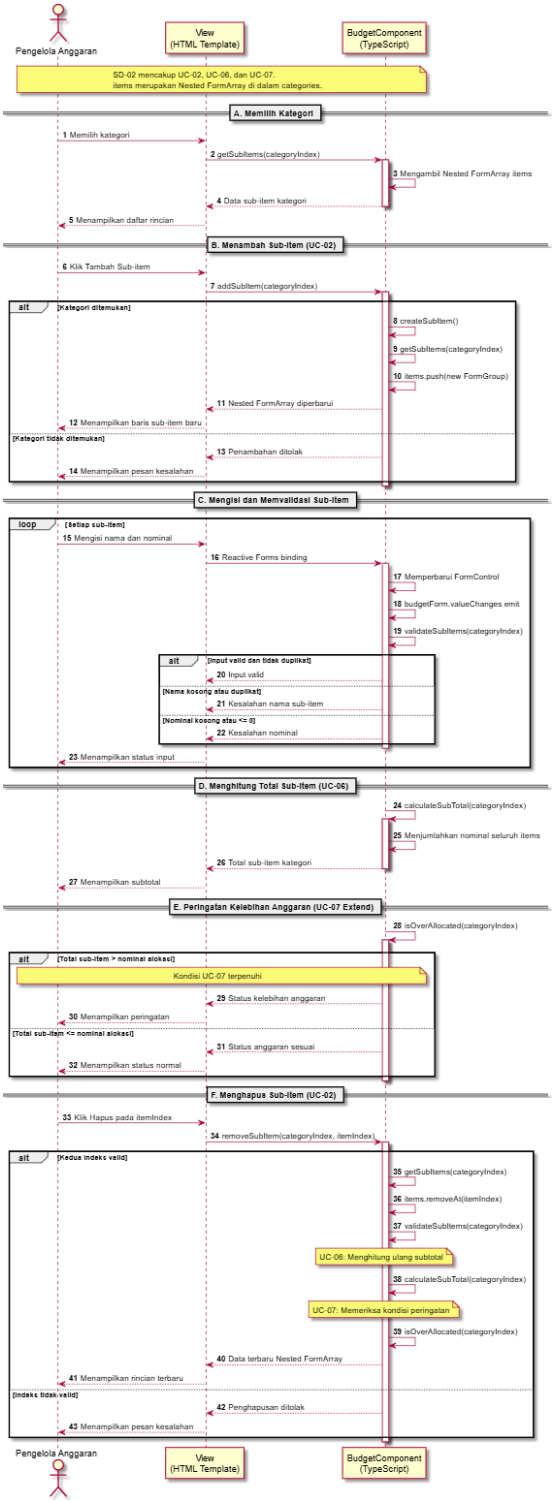


Gambar 3.1 Sequence Diagram Mengelola Alokasi Anggaran

3.2.2 SD-02 — Mengelola Rincian Sub-item

Sequence Diagram ini menggambarkan proses pengelolaan rincian pengeluaran pada setiap kategori. Pengguna dapat menambahkan atau menghapus sub-item beserta nominalnya.

Setiap perubahan data akan memicu perhitungan subtotal dan validasi terhadap nominal alokasi kategori. Apabila total sub-item melebihi nominal alokasi, sistem menampilkan peringatan kelebihan anggaran.

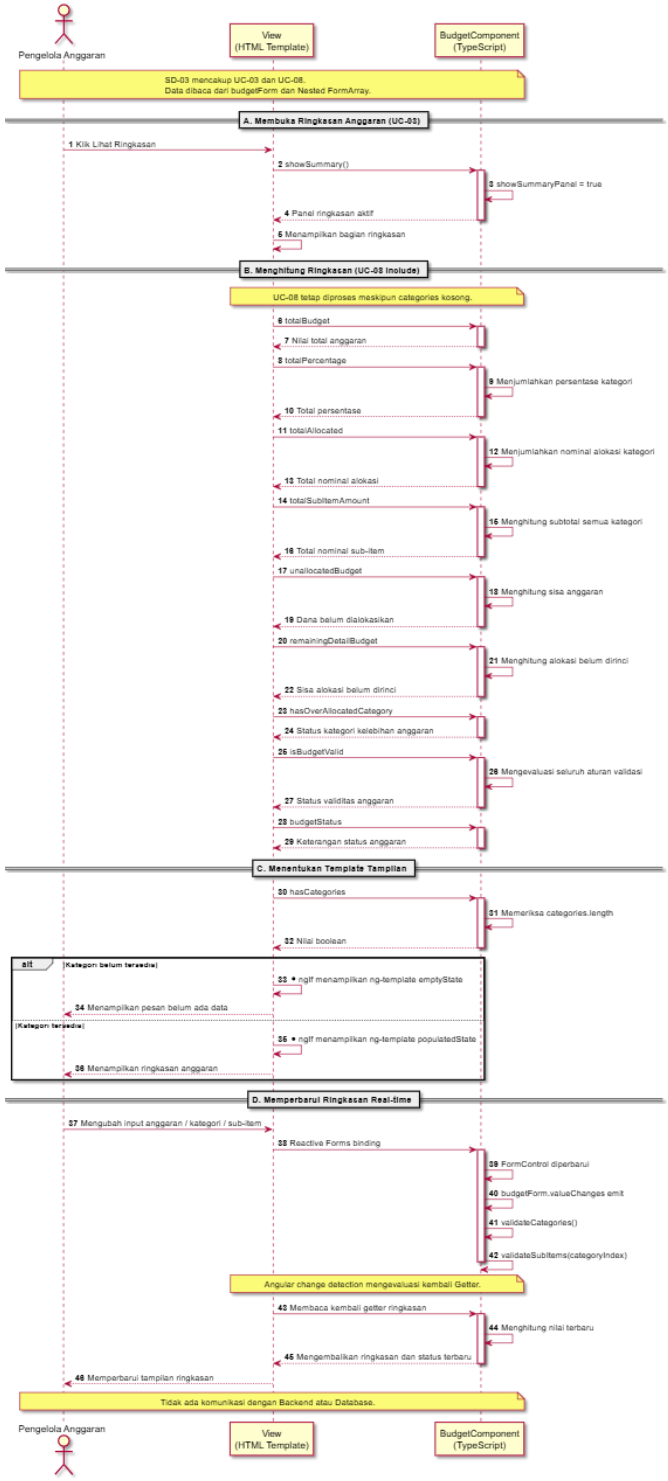


Gambar 3.2 Sequence Diagram Mengelola Rincian Sub-item

3.2.3 SD-03 — Melihat Ringkasan Anggaran

Sequence Diagram ini menggambarkan proses sistem dalam mengolah dan menampilkan ringkasan keseluruhan anggaran.

Sistem menghitung total persentase, total nominal alokasi, total sub-item, serta sisa anggaran. Selanjutnya, sistem menentukan kondisi tampilan berdasarkan ketersediaan data, yaitu Empty State apabila belum terdapat kategori dan Populated State apabila data kategori tersedia.



Gambar 3.3 Sequence Diagram Melihat Ringkasan Anggaran

3.3 Kesimpulan Perancangan Interaksi Sistem

Berdasarkan Use Case Description dan Sequence Diagram yang telah dibuat, aplikasi Smart Budget Allocation Planner memiliki tiga proses utama, yaitu pengelolaan alokasi anggaran, pengelolaan rincian sub-item, dan penampilan ringkasan anggaran.

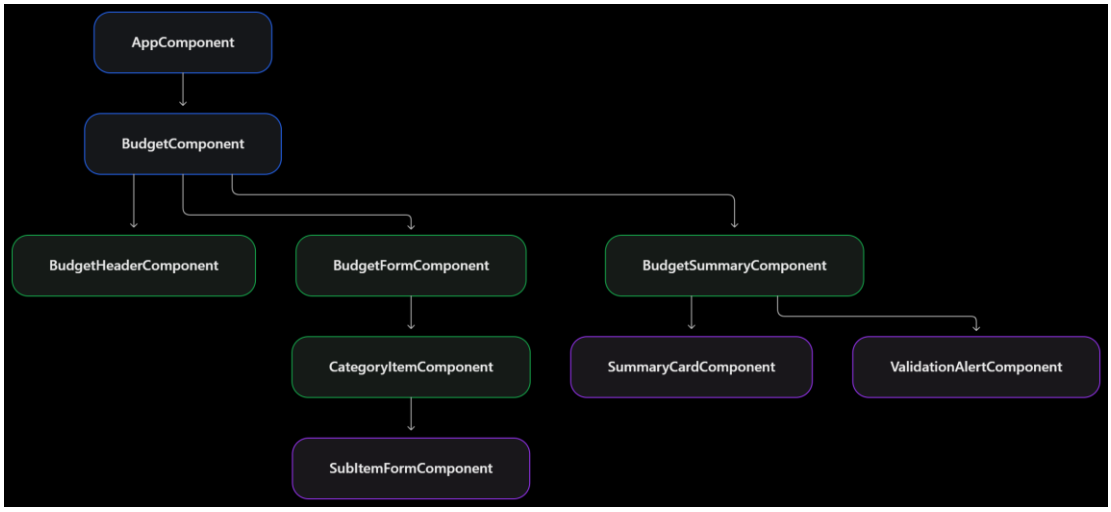
Ketiga proses tersebut saling terhubung melalui mekanisme validasi dan perhitungan otomatis. Rancangan interaksi ini menjadi dasar dalam menentukan arsitektur komponen dan implementasi aplikasi menggunakan framework Angular.

4. COMPONENT ARCHITECTURE & DATA MAPPING

4.1 Component Architecture

Aplikasi Smart Budget Allocation Planner dibangun menggunakan pendekatan arsitektur berbasis komponen pada framework Angular. Aplikasi dibagi menjadi beberapa komponen yang memiliki tanggung jawab masing-masing untuk meningkatkan modularitas, keterbacaan, dan kemudahan pemeliharaan kode program.

Komponen utama dalam aplikasi ini adalah `BudgetComponent`, yang berfungsi sebagai pusat pengelolaan state dan logika aplikasi. Komponen ini bertanggung jawab mengelola Reactive Forms, FormArray, Nested FormArray, validasi lintas baris, kalkulasi alokasi anggaran, serta pengolahan ringkasan anggaran.



Gambar 4.1 Component Tree Smart Budget Allocation Planner

Hierarki komponen aplikasi terdiri dari sembilan komponen dengan fungsi sebagai berikut:

Tabel 4.1 Hierarki dan Tanggung Jawab Komponen

No.	Nama Komponen	Tanggung Jawab
1	AppComponent	Sebagai root component aplikasi.
2	BudgetComponent	Mengelola state, form, validasi, kalkulasi, dan

		komunikasi data utama.
3	BudgetHeaderComponent	Menampilkan identitas dan judul aplikasi.
4	BudgetFormComponent	Menampilkan form total anggaran dan daftar kategori.
5	CategoryItemComponent	Menampilkan setiap kategori beserta persentase dan nominal alokasinya.
6	SubItemFormComponent	Menampilkan dan mengelola rincian pengeluaran pada setiap kategori.
7	BudgetSummaryComponent	Menampilkan ringkasan keseluruhan anggaran.
8	SummaryCardComponent	Menampilkan informasi ringkasan dalam bentuk kartu.
9	ValidationAlertComponent	Menampilkan pesan dan status validasi anggaran.

Berdasarkan hierarki tersebut, `BudgetComponent` bertindak sebagai Parent Component yang mengelola keseluruhan data aplikasi, sedangkan komponen lainnya bertindak sebagai Child Component yang bertanggung jawab terhadap tampilan dan interaksi pengguna.

4.2 View-Controller Mapping

Pemetaan View-Controller dilakukan dengan menghubungkan elemen HTML Template terhadap property dan method pada Component TypeScript.

Setiap interaksi pengguna, seperti penambahan kategori, penghapusan kategori, penambahan sub-item, dan penghapusan sub-item, diteruskan kepada `BudgetComponent` untuk diproses.

Pengelolaan data dilakukan menggunakan Angular Reactive Forms dengan struktur FormGroup dan FormArray, sehingga perubahan input dapat diproses secara dinamis tanpa memerlukan backend atau database.

Adapun pemetaan elemen View terhadap Controller pada aplikasi Smart Budget Allocation Planner adalah sebagai berikut.

Tabel 4.2 View-Controller Mapping

No.	Komponen View	Elemen View / HTML	Property atau Method Controller
1	BudgetFormComponent	Input total anggaran	`budgetForm/totalBudget`
2	BudgetFormComponent	Tombol Tambah Kategori	`addCategory()`
3	CategoryItemComponent	Input nama kategori	`categoryGroup.name`
4	CategoryItemComponent	Input persentase kategori	`categoryGroup.percentage`
5	CategoryItemComponent	Tombol Hapus Kategori	`removeCategory(index)`
6	SubItemFormComponent	Tombol Tambah Sub-item	`addSubItem(categoryIndex)`
7	SubItemFormComponent	Input nama sub-item	`itemGroup.name`
8	SubItemFormComponent	Input nominal sub-item	`itemGroup.amount`
9	SubItemFormComponent	Tombol Hapus Sub-item	`removeSubItem(categoryIndex, itemIndex)`
10	BudgetSummaryComponent	Total persentase	`totalPercentage`

11	BudgetSummaryComponent	Total alokasi	`totalAllocated`
12	BudgetSummaryComponent	Total sub-item	`totalSubItemAmount`
13	BudgetSummaryComponent	Sisa anggaran belum dialokasikan	`unallocatedBudget`
14	BudgetSummaryComponent	Kondisi data kategori	`hasCategories`
15	ValidationAlertComponent	Pesan validasi	`budgetStatus`
16	BudgetSummaryComponent	Status validasi anggaran	`isBudgetValid`

Berdasarkan tabel tersebut, setiap elemen HTML memiliki hubungan dengan property atau method pada Controller. Misalnya, ketika pengguna menekan tombol Tambah Kategori, komponen View akan meneruskan event kepada `BudgetComponent` untuk menjalankan method `addCategory()`.

Dengan demikian, pengolahan data tetap dilakukan pada Component TypeScript, sedangkan HTML Template bertanggung jawab untuk menerima input dan menampilkan hasil pengolahan data.

4.3 Data Mapping Antar-Komponen

Komunikasi data antar-komponen pada aplikasi Smart Budget Allocation Planner menggunakan mekanisme `@Input`, `@Output`, dan `EventEmitter`.

Mekanisme `@Input` digunakan untuk mengirimkan data dari komponen induk (Parent Component) menuju komponen anak (Child Component). Sementara itu, `@Output` dan `EventEmitter` digunakan untuk mengirimkan event atau informasi tindakan dari komponen anak menuju komponen induk.

Adapun pemetaan komunikasi data antar-komponen adalah sebagai berikut.

Tabel 4.3 Data Mapping Antar-Komponen

No.	Parent Component	Child Component	@Input (Data Dikirim)
1	AppComponent	BudgetComponent	Tidak ada data khusus.
2	BudgetComponent	BudgetFormComponent	`budgetForm`, `categories`
3	BudgetFormComponent	CategoryItemComponent	`categoryGroup`, `categoryIndex`, `categoryAmount`
4	CategoryItemComponent	SubItemFormComponent	`items`, `categoryIndex`, `subTotal`, `isOverAllocated`
5	BudgetComponent	BudgetSummaryComponent	`totalBudget`, `totalPercentage`, `totalAllocated`, `totalSubItemAmount`, `unallocatedBudget`, `remainingDetailBudget`, `hasCategories`, `isBudgetValid`, `budgetStatus`
6	BudgetSummaryComponent	SummaryCardComponent	`title`, `value`
7	BudgetSummaryComponent	ValidationAlertComponent	`status`, `message`

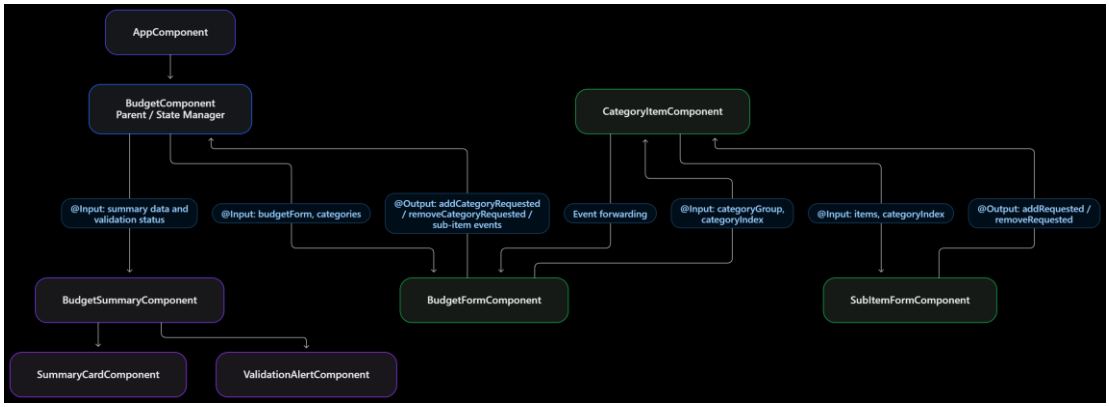
Selain pengiriman data menggunakan `@Input`, komunikasi dari Child Component menuju Parent Component dilakukan melalui `@Output` dan `EventEmitter`.

Tabel 4.4 Pemetaan Event Antar-Komponen

No.	Child Component	EventEmitter (@Output)	Parent Penerima	Method Controller

1	BudgetFormComponent	`addCategoryRequested`	BudgetComponent	`addCategory()`
2	BudgetFormComponent	`removeCategoryRequested`	BudgetComponent	`removeCategory(index)`
3	CategoryItemComponent	`addSubItemRequested`	BudgetFormComponent	Diteruskan ke `addSubItem(categoryIndex)`
4	CategoryItemComponent	`removeSubItemRequested`	BudgetFormComponent	Diteruskan ke `removeSubItem(categoryIndex, itemIndex)`
5	SubItemFormComponent	`addRequested`	CategoryItemComponent	Diteruskan ke `addSubItemRequested`
6	SubItemFormComponent	`removeRequested`	CategoryItemComponent	Diteruskan ke `removeSubItemRequested`

Event yang berasal dari komponen anak diteruskan melalui komponen induk hingga diterima oleh `BudgetComponent` untuk diproses. Dengan demikian, seluruh perubahan data tetap dikendalikan oleh satu pusat pengelolaan state aplikasi.



Gambar 4.2 Diagram Komunikasi Data Antar-Komponen

Berdasarkan pemetaan tersebut, aplikasi menerapkan pola komunikasi Parent-Child Component untuk mengatur pertukaran data dan interaksi antar-komponen. `BudgetComponent` bertindak sebagai pusat pengelolaan state, sedangkan komponen anak menerima data menggunakan `@Input` dan mengirimkan event menggunakan `@Output` dan `EventEmitter`.

Pendekatan ini menjadikan arsitektur aplikasi lebih modular, terstruktur, dan konsisten dengan rancangan Use Case Diagram serta Sequence Diagram yang telah dibuat.